



Universidad de Carabobo  
Facultad Experimental de Ciencias y Tecnología  
Arquitectura del Computador  
Sección 1



# Informe

## Práctica de Laboratorio 3

**Integrantes:**

Penélope Robledo, C.I. 32.482.938

Nathalia Noguera, C.I. 32.756.046

**Profesor:**

José Canache

Naguanagua, agosto de 2026

# **1. Explique cómo se organiza la memoria cuando un sistema utiliza memory-mapped I/O. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones lw y sw?**

Cuando un sistema utiliza memory-mapped I/O, lo que sucede es que el espacio de direcciones del procesador deja de estar reservado exclusivamente para la memoria RAM y pasa a compartirse entre la memoria principal y los registros de control de los dispositivos periféricos. En lugar de que el procesador tenga dos espacios de direccionamiento separados (uno para memoria y otro para dispositivos), existe un único mapa de direcciones de 32 bits, y dentro de ese mapa se reserva un rango específico para que cada dispositivo exponga sus registros como si fueran posiciones de memoria comunes.

Esto es lo que se simuló en los ejercicios de la práctica, en el sensor de luminosidad se reservaron tres palabras en la sección .data que son LuzControl, LuzEstado y LuzDatos, y en el módulo de tensión arterial se reservaron cuatro palabras nombradas TensionControl, TensionEstado, TensionSistol y TensionDiastol. Desde el punto de vista del ensamblador estas etiquetas no tienen absolutamente nada que las distinga de una variable normal, pero conceptualmente representan los registros internos de un dispositivo de hardware, que en un sistema real estarían soldados a un chip externo al procesador y conectados mediante un bus, no dentro de un banco de memoria DRAM.

En los sistemas MIPS32 reales esta región reservada suele ubicarse en la parte alta del espacio de direcciones, típicamente en la zona no cacheable llamada KSEG1, que en la arquitectura MIPS clásica comienza en 0xA0000000, y dentro del simulador MARS específicamente el bloque de memoria de E/S por defecto arranca en 0xFFFF0000. La razón de colocar los dispositivos en direcciones altas y no mezclados entre las direcciones bajas donde vive el código y los datos del programa es evitar cualquier posibilidad de que una escritura accidental sobre un arreglo o una variable termine sobrescribiendo un registro de control de un dispositivo, y viceversa, que una lectura fuera de rango de un arreglo termine leyendo el estado de un sensor. En la práctica esto

se resuelve reservando un segmento completo, separado físicamente del segmento .data y del segmento .text, para que el decodificador de direcciones del hardware pueda distinguir con solo mirar los bits más significativos si la dirección corresponde a RAM o a un periférico.

En cuanto a la implicación sobre las instrucciones lw y sw, la consecuencia más importante es que en MIPS32 no existen instrucciones dedicadas para hacer entrada y salida, por lo que lw y sw terminan cumpliendo una doble función, la de mover datos entre registros y memoria, y la de comunicarse con los dispositivos. Esto se puede ver de forma literal en el código de ambos ejercicios, cuando en InicializarSensorLuz se ejecuta `sw $t0, 0($t1)` después de cargar la dirección de LuzControl en \$t1, en realidad no se está guardando un dato en una celda de RAM cualquiera, se está enviando la orden de inicialización al sensor, y cuando en el bucle de espera se ejecuta `lw $t2, 0($t1)` sobre la dirección de LuzEstado, no se está leyendo un valor estático que el programa escribió antes, sino que se está consultando en tiempo real el estado que el dispositivo va actualizando por su cuenta.

Lo mismo tiene lugar en procesarMedicion, donde `sw $t0, 0($t1)` sobre TensionControl arranca la medición, y el ciclo espera con `lw $t2, 0($t1)` sobre TensionEstado va vigilando el registro hasta que el dispositivo cambia su valor de 0 a 1. El detalle técnico más delicado, aunque MARS no lo simule porque trabaja sobre memoria estática y no sobre hardware físico, es que en un procesador real estas direcciones deben marcarse como no cacheables, porque si el valor de LuzEstado o TensionEstado quedara guardado en una línea de caché, el bucle de espera seguiría leyendo el valor viejo almacenado en la caché en lugar del valor real y actualizado que el dispositivo está escribiendo en su registro físico, y el programa quedaría atrapado en un ciclo infinito sin que el sensor haya fallado realmente. Por esa razón, cuando se trabaja con memory-mapped I/O, cada lw y cada sw dirigido a esa región deben entenderse no como accesos a memoria en el sentido tradicional, sino como órdenes o consultas de hardware, aunque su sintaxis en ensamblador sea idéntica a la de cualquier otro acceso a memoria.

## **2. ¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O?**

La diferencia central entre ambos enfoques está en si el espacio de direcciones de los dispositivos comparte o no el mismo mapa que la memoria principal. En memory-mapped I/O, que fue utilizado para estos ejercicios, los registros del sensor de luminosidad y del módulo de tensión arterial ocupan direcciones dentro del mismo espacio de 32 bits en el que también se hallan las variables normales del programa, por eso se pudieron declarar con `.word` dentro de `.data` igual que cualquier otro dato, y se accede a ellos con las mismas instrucciones de carga y almacenamiento que se usan para leer y escribir cualquier variable.

En la entrada y salida por puertos, en cambio, existe un espacio de direcciones completamente separado y de menor tamaño, exclusivo para dispositivos, al que solo se puede acceder mediante instrucciones especiales del procesador, como `in` y `out` en la arquitectura x86, y ese espacio de puertos nunca se solapa ni se confunde con el de la memoria aunque numéricamente coincidan los valores de dirección, porque el procesador activa una línea de control distinta según si la operación es de memoria o de E/S.

La ventaja principal del enfoque por puertos es que mantiene completamente separado el direccionamiento de memoria del de los dispositivos, lo que evita que el programador tenga que sacrificar parte del espacio de direcciones de RAM para reservarlo a periféricos, y en teoría permite decodificadores de dirección más simples porque el espacio de puertos suele ser mucho más pequeño, por ejemplo 16 bits en x86 en vez de 32. El inconveniente es que obliga a que el conjunto de instrucciones del procesador incluya operaciones dedicadas solamente para E/S, que no pueden combinarse libremente con los modos de direccionamiento ni con las instrucciones aritméticas y lógicas normales, por lo que no se puede sumar directamente el contenido de un puerto con un registro, primero hay que traerlo con `IN` a un registro general y luego operar.

Memory-mapped I/O se desempeña de manera distinta, siendo su ventaja que cualquier ins-

trucción que trabaje con memoria automáticamente puede trabajar con un dispositivo, sin necesitar instrucciones nuevas, y esto se comprueba en el código de ambos algoritmos, donde el mismo beq que se usa para comparar registros comunes también se usa para comparar el valor leído de LuzEstado o TensionEstado con cero dentro del ciclo de espera, y donde el mismo li que carga una constante en un registro es el que prepara el valor 1 que luego se envía a LuzControl o TensionControl. Una dificultad que se presenta en memory-mapped I/O es lo que se explicó anteriormente, se debe reservar una porción del espacio de direcciones para los dispositivos, lo que reduce el rango disponible para memoria, y exige que ese rango quede fuera del alcance de la caché para que las lecturas siempre reflejen el estado real del hardware.

MIPS32 emplea principalmente memory-mapped I/O porque su diseño se basa en una arquitectura RISC de carga y almacenamiento en la que solamente lw y sw pueden acceder a memoria y todas las demás instrucciones trabajan exclusivamente sobre registros. Esta decisión de diseño busca mantener el conjunto de instrucciones lo más reducido y sencillo de decodificar posible, así que agregar instrucciones especiales de entrada y salida iría en contra de ese principio, porque implicaría un formato de instrucción adicional, un decodificador adicional en la unidad de control, y una ruta de datos separada solo para mover información entre los puertos y los registros.

Al reutilizar lw y sw para comunicarse con los dispositivos, MIPS32 evita todo ese costo adicional en hardware y en complejidad del compilador, y es precisamente por esto que en la práctica no fue necesario usar ninguna instrucción fuera de las básicas. Con la, lw, sw, li, beq y jr fue suficiente para inicializar los dispositivos, determinar su estado y recuperar los datos medidos.

### **3. En un sistema con memory-mapped I/O: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?**

Si dos dispositivos llegaran a compartir, aunque sea parcialmente, el mismo rango de direcciones dentro del mapa de memoria, el problema más inmediato es que una sola instrucción lw o sw terminaría afectando o consultando a más de un dispositivo a la vez sin que el procesador tenga forma de saber que eso está pasando. Para ejemplificar, si en esta práctica LuzControl y TensionControl hubieran quedado ubicados exactamente en la misma dirección de memoria por un error del mapa de direcciones, al ejecutar sw \$t0, 0(\$t1) para inicializar el sensor de luminosidad, esa misma escritura también daría inicio a una medición de tensión arterial sin que el programa lo haya pedido, y de igual forma, el bucle de espera de un dispositivo podría terminar leyendo el registro de estado del otro, dando una falsa señal de que la lectura ya está lista cuando en realidad corresponde a un proceso completamente distinto.

A nivel de hardware el problema es incluso más grave que una simple confusión lógica, porque si ambos dispositivos están físicamente conectados al bus y los dos intentan responder a la misma dirección al mismo tiempo, se produce lo que se conoce como contención de bus, con dos circuitos tratando de imponer un voltaje distinto sobre la misma línea de datos simultáneamente, lo que en un sistema real puede generar lecturas corruptas, comportamiento eléctrico indefinido, e incluso daño físico al hardware en casos extremos.

Este choque se evita mediante un mapa de direcciones bien definido y verificado antes de escribir una sola línea de código, en el que a cada dispositivo se le asigna un rango de direcciones exclusivo y sin interferir con el de cualquier otro periférico o con el de la memoria RAM. Este trabajo lo realiza automáticamente el ensamblador de MARS, porque al declarar LuzControl, LuzEstado y LuzDatos en un archivo, y TensionControl, TensionEstado, TensionSistol y TensionDiastol en otro, cada etiqueta recibe una dirección distinta dentro del segmento .data, sin que se tenga que calcular direcciones absolutas a mano, así que el propio ensamblador ya garantiza que no haya solapamiento entre las variables declaradas.

En un sistema con hardware real, sin embargo, esa garantía no la da un ensamblador sino un circuito llamado decodificador de direcciones, que consiste en compuertas lógicas o comparadores que examinan los bits más significativos de la dirección que sale por el bus y, dependiendo de en qué rango esté, activan una única línea de selección de chip correspondiente a un solo dispositivo. Mientras el diseñador del sistema respete que cada rango asignado a un dispositivo no se cruce con el de otro, y que el decodificador esté correctamente cableado para reconocer esos rangos, es imposible que dos dispositivos respondan a la misma dirección a la vez, porque solo uno de ellos recibirá la señal de selección activa en cualquier instante dado. Esto es justamente lo que permite que, aunque conceptualmente el sensor de luminosidad y el módulo de tensión arterial pudieran convivir en el mismo sistema físico, sus registros de control y estado permanezcan apartados sin interferir uno con el otro.

#### **4. ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?**

El memory-mapped I/O facilita el diseño del conjunto de instrucciones porque elimina por completo la necesidad de que el procesador tenga un mecanismo diferenciado para intercambiar datos con los dispositivos, todo lo que ya existe para trabajar con memoria sirve automáticamente para trabajar con hardware externo. En los algoritmos desarrollados esto se aprovecha de principio a fin, la inicialización de un dispositivo se resuelve con la misma pareja de instrucciones la y sw que se usaría para guardar cualquier variable, en InicializarSensorLuz se hace la \$t1, LuzControl seguido de sw \$t0, 0(\$t1), y en procesarMedicion se hace exactamente lo mismo con la \$t1, TensionControl. La evaluación del estado del dispositivo, que es la parte más delicada de ambos algoritmos porque implica esperar activamente a que el hardware termine su trabajo, se lleva a cabo con instrucciones estándar de comparación y salto condicional, el bucle espera usa lw \$t2, 0(\$t1) para traer el estado actual y beq \$t2, \$zero, espera para decidir si se sigue esperando, sin que exista ninguna instrucción especializada de tipo “esperar a dispositivo” o “consultar puerto”.

La recuperación de los datos finales, como la luminosidad leída o los valores de tensión sistólica y diastólica, se hace con un simple lw hacia \$v0 y \$v1, que son los mismos registros que cualquier función usaría para devolver un resultado numérico ordinario. Como consecuencia, el procesador no necesita un formato de instrucción adicional, un opcode adicional, una unidad de control más compleja, ni rutas de datos separadas para manejar E/S, todo el trabajo lo hace la misma unidad de carga y almacenamiento que ya existe para acceder a memoria.

Si en su lugar se hubiera optado por entrada y salida por puertos, habría sido necesario incorporar al menos dos instrucciones completamente nuevas al repertorio de MIPS32, típicamente llamadas algo como in y out, cuya función sería mover un dato entre un registro del procesador y un puerto identificado por un número. Estas instrucciones necesitarían su propio formato,



probablemente uno que incluya un campo separado para el número de puerto en lugar de un campo de dirección de memoria completo de 32 bits, ya que el espacio de puertos suele ser mucho más reducido. También sería indispensable agregar al bus de control una línea adicional que le indique al resto del sistema si el ciclo de bus actual corresponde a un acceso de memoria o a un acceso de E/S, para que el decodificador de direcciones sepa a cuál de los dos espacios pertenece la operación, algo que en memory-mapped I/O no hace falta porque toda la información ya está implícita en la propia dirección. Sumado a esto, sería necesario que el compilador y el propio programador tuvieran presente esta distinción constantemente, porque un valor leído desde un puerto no podría operarse directamente con instrucciones aritméticas como add o sub, primero tendría que moverse a un registro general con la instrucción especial de entrada, agregando pasos y complejidad justo en el tipo de tareas que en esta práctica se resolvieron de forma directa, como comparar el estado leído o mover el dato leído a \$v0 sin ningún paso intermedio.

## **5. ¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?**

Cuando el procesador ejecuta una instrucción como lw o sw dirigida a una dirección que corresponde a un dispositivo, tal como cuando se ejecuta sw \$t0, 0(\$t1) con \$t1 apuntando a LuzControl, o lw \$t2, 0(\$t1), el dato contenido en el registro fuente se coloca sobre el bus de datos para que viaje junto con la dirección y la señal de control, mientras que en una lectura como lw, el bus de datos queda a la espera de que el dispositivo o la memoria coloquen el valor solicitado sobre esas mismas líneas para que el procesador lo capture y lo guarde en el registro destino.

La forma en que el hardware sabe si debe atender esa dirección con la memoria RAM o con un dispositivo periférico es a través del decodificador de direcciones previamente mencionado. Este decodificador está observando permanentemente los bits más significativos del bus de direcciones, y dependiendo del rango en el que resida el valor presente en el bus, activa una única línea de selección de chip, ya sea la que corresponde al controlador de memoria RAM o al controlador del dispositivo específico.

En el caso particular de esta práctica, si LuzControl, LuzEstado y LuzDatos hubieran sido implementados como un dispositivo físico real conectado a un procesador MIPS32, sus tres direcciones habrían estado dentro del rango reservado para entrada y salida, probablemente en la región KSEG1 no cacheable, y el decodificador habría reconocido esos bits altos como pertenecientes al controlador del sensor de luminosidad en lugar de al controlador de memoria, encaminando la señal hacia el chip del sensor para que fuera este quien respondiera colocando el valor correspondiente en el bus de datos. Esto también aplicaría para TensionControl, TensionEstado, TensionSistol y TensionDiastol respecto al controlador del módulo de tensión arterial.

En MARS, como es un entorno simulado sobre software y no existe un bus físico ni un decodificador real, todo este proceso queda oculto, el simulador simplemente trata esas direcciones

como cualquier otra posición dentro de su modelo de memoria y responde con el valor almacenado en esa celda, pero cada lw y cada sw dirigido a las variables que representan los registros del sensor y del módulo de tensión debe entenderse como si en un sistema real hubiera creado exactamente esta secuencia de eventos en el bus, la colocación de una dirección, la activación de una señal de control, y la respuesta de un dispositivo distinto a la memoria principal.

## **6. ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?**

En lo que se refiere a esta práctica dentro del simulador MARS, la respuesta es sí sin ningún tipo de restricción, porque MARS no implementa modos de privilegio ni protección de memoria de ningún tipo, todo el espacio de direcciones es plano y accesible por igual desde cualquier parte del programa. Esto se puede comprobar directamente en el comportamiento de los algoritmos desarrollados, en InicializarSensorLuz la instrucción `sw $t0, 0($t1)` escribe sobre LuzControl sin que exista ninguna verificación previa de que tenga permiso para hacerlo, y en procesarMedicion ocurre exactamente igual con TensionControl.

El simulador ejecuta la instrucción tal cual está escrita, sin comprobar si quien la ejecuta debería o no tener acceso a esa dirección de memoria. Esto tiene sentido porque lo que se busca con esta herramienta es observar la mecánica de la comunicación con un dispositivo mapeado en memoria, no lidiar con capas adicionales de un sistema operativo.

No obstante, en un sistema operativo real las cosas funcionan de una manera completamente distinta, y ahí es donde entra la operación en modo dual, que quiere decir, la existencia de al menos dos niveles de privilegio en el procesador, comúnmente llamados modo usuario y modo kernel o modo supervisor. En la arquitectura MIPS32 esto se controla mediante el coprocesador 0, específicamente a través del registro Status, que contiene un bit llamado KU (Kernel/User) que indica en qué modo se encuentra ejecutando el procesador en un momento dado.

Cuando el procesador está en modo usuario, el espacio de direcciones virtuales al que puede acceder queda restringido al segmento llamado KUSEG, que ocupa la mitad inferior del espacio de 32 bits, desde 0x00000000 hasta 0x7FFFFFFF, y es en esta zona donde se ubicarían el código, los datos y la pila de cualquier programa de aplicación normal. Los segmentos KSEG0 y KSEG1, que ocupan la mitad superior del espacio de direcciones, solo son accesibles cuando el procesador se encuentra en modo kernel, y es en KSEG1, la región no cacheada y sin traducción de direcciones, donde en un sistema MIPS se ubican los registros de los dispositivos mapeados en

memoria, entre ellos los que en esta práctica corresponderían a LuzControl, LuzEstado, LuzDatos, TensionControl, TensionEstado, TensionSistol y TensionDiastol si estuvieran implementados sobre hardware físico.

Si un programa que se está ejecutando en modo usuario intentara ejecutar un lw o un sw con una dirección perteneciente a KSEG1, el procesador no llevaría a cabo la operación, sino que generaría una excepción de tipo error de dirección, conocida en MIPS como AdEL (Address Error Exception) para el caso de lectura o AdES para el caso de escritura, que transfiere inmediatamente el control al manejador de excepciones del sistema operativo en lugar de completar el acceso solicitado.

Aparte del control por segmento que ofrece la propia organización del espacio de direcciones de MIPS, en procesadores que cuentan con una unidad de gestión de memoria completa, la MMU, existe un mecanismo aún más flexible, que consiste en marcar cada página de memoria dentro de la tabla de páginas con bits de permisos, entre ellos un bit de privilegio que indica si esa página puede ser accedida desde modo usuario o si está reservada exclusivamente para modo kernel. Con este mecanismo, el sistema operativo puede definir qué páginas contienen registros de dispositivos y marcarlas como inaccesibles para cualquier proceso de usuario, de manera que cualquier intento de acceso desde ese nivel produzca una excepción de protección gestionada por el propio sistema operativo, que normalmente termina el proceso.

La forma en que un programa de aplicación normal sí puede llegar a interactuar con un dispositivo, sin tener acceso directo a sus registros, es a través de llamadas al sistema o syscalls, que son el único mecanismo controlado para que el modo usuario le pida a modo kernel que realice una operación en su nombre. Cuando un programa de usuario necesita leer del sensor de luminosidad o de un módulo de tensión arterial, no ejecuta él mismo el lw sobre LuzEstado o TensionEstado, sino que hace un syscall que causa una excepción controlada, el procesador cambia automáticamente a modo kernel, y es el controlador de dispositivo, el driver, que forma parte del sistema operativo, quien realmente ejecuta las instrucciones lw y sw contra los registros del dispositivo, validando previamente los parámetros recibidos para evitar accesos indebidos o mal formados.

La estructura de los códigos en esta práctica, donde el main nunca toca directamente Luz-

Control ni TensionControl, sino que delega ese trabajo a InicializarSensorLuz, LeerLuminosidad y procesarMedicion, imita a pequeña escala esta separación entre quien pide la operación y quien realmente tiene acceso al hardware, aunque en este caso la separación sea solamente una convención de buena organización del código y no una barrera de protección impuesta por el hardware, ya que dentro de MARS cualquier parte del programa podría teóricamente acceder directamente a esas direcciones sin pasar por las partes del programa correspondientes.

## **7. ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?**

Los algoritmos correspondientes a los ejercicios de esta práctica realizan la sincronización con el dispositivo mediante una espera activa, lo que se ve de forma clara en ambos bucles etiquetados como espera. Mientras el sensor de luminosidad se está preparando o mientras el módulo de tensión arterial está tomando la medición, el procesador no hace absolutamente nada útil, se limita a preguntar una y otra vez, a la máxima velocidad posible del reloj, si el dispositivo ya terminó. Para estos ejercicios donde el dispositivo simulado responde de inmediatamente esto no es un verdadero problema, pero si se piensa en el caso de TensionEstado, un dispositivo físico de medición de tensión arterial normalmente necesita varios segundos para inflar el brazalete, tomar las lecturas y desinflarlo, y mantener al procesador dando vueltas en un ciclo `beq` durante varios segundos completos sería un desperdicio enorme de tiempo que podría estar utilizándose para ejecutar cualquier otra tarea del sistema.

La técnica más usada para resolver este problema es reemplazar la revisión activa por un esquema dirigido por interrupciones. En lugar de que el procesador pregunte constantemente por el estado del dispositivo, es el propio dispositivo el que le avisa al procesador cuando ha terminado su trabajo, activando una línea de interrupción física conectada al procesador. En MIPS32 esto se gestiona a través del registro `Cause` y del registro `Status` del coprocesador 0, específicamente mediante los bits de máscara de interrupción `IM` que permiten habilitar o deshabilitar distintas fuentes de interrupción.

Con este esquema, la subrutina equivalente a `InicializarSensorLuz` o a `procesarMedicion` no entraría en un bucle, sino que después de escribir en `LuzControl` o en `TensionControl` simplemente cedería el control del procesador, ya sea devolviendo la ejecución al sistema operativo para que este programe otra tarea en la CPU, o ejecutando una instrucción de bajo consumo que detiene el pipeline hasta que ocurra un evento externo. Cuando el sensor termina de inicializarse o cuando el módulo de tensión arterial completa la medición, el dispositivo activa físicamente la línea de interrupción, el procesador automáticamente guarda su contexto de ejecución actual, salta a la dirección del manejador de interrupciones correspondiente, y es dentro de ese ma-

nejador donde finalmente se ejecutaría el lw sobre LuzDatos, o los dos lw sobre TensionSistol y TensionDiastol, exactamente igual que se hace al final de LeerLuminosidad y al final de procesarMedicion en el código, solo que ahora esa lectura ocurre en respuesta a un evento y no como resultado de haber estado preguntando en un ciclo sin parar.

Otra técnica complementaria, especialmente útil cuando se trata de transferencias de bloques grandes de datos en lugar de una sola palabra, es el uso de un controlador de acceso directo a memoria, conocido como DMA. Con DMA, el procesador únicamente configura la operación, indicándole al controlador la dirección de origen, la dirección de destino y la cantidad de datos a transferir, y a partir de ahí es el propio controlador DMA quien mueve los datos entre el dispositivo y la memoria principal sin que la CPU tenga que intervenir en cada palabra transferida, liberando al procesador durante toda la duración de la transferencia y notificándolo mediante una interrupción solo al culminar. Aunque en el caso del sensor de luminosidad y del módulo de tensión arterial las cantidades de datos son mínimas, un solo valor entero para la luminosidad y dos valores enteros para la tensión sistólica y diastólica, en dispositivos que manejan flujos continuos de datos, como un sensor de audio o una interfaz de red, el DMA resulta indispensable para evitar que el procesador quede atado a supervisar cada palabra individual que entra o sale del dispositivo.

Por último, existe también un enfoque que combina las dos estrategias, conocido como polling adaptativo, en el cual el procesador realiza una inspección activa únicamente durante un intervalo breve al inicio, apostando a que ciertos dispositivos suelen responder casi de inmediato, y si tras ese intervalo el dispositivo todavía no está listo, en lugar de continuar preguntando indefinidamente, el procesador cambia a un esquema de interrupciones para el resto de la espera. Esta técnica busca aprovechar lo mejor de cada uno, evitando la sobrecarga de configurar y atender una interrupción para operaciones extremadamente rápidas, pero sin llegar a desperdiciar ciclos de reloj de forma indefinida en operaciones que tardan más de lo esperado, como podría ser el caso de una medición de tensión arterial que, por alguna razón física, tome más tiempo del habitual en completarse.



## 8. Análisis y Discusión de los Resultados.

Para el análisis de esta práctica, procedemos a verificar que ambos algoritmos implementan de forma correcta el protocolo de comunicación con los registros simulados de cada dispositivo, que el mapa de memoria de cada uno queda organizado sin ambigüedades, y que el comportamiento del programa ante cada estado posible del hardware corresponde exactamente a lo que especifica el enunciado. Se presentará la comprobación de que el sensor de luminosidad y el módulo de tensión arterial fueron correctamente modelados como dispositivos mapeados en memoria, además de que las subrutinas desarrolladas manejan de manera adecuada las tres fases propias de este tipo de comunicación, la orden de control, la espera del estado y la lectura final del dato.

### a. Verificación del mapa de memoria asignado a cada dispositivo.

Lo primero que se comprobó, apoyándose en la ventana de memoria de MARS, es dónde queda realmente ubicado cada registro simulado dentro del espacio de direcciones del programa. Al inspeccionar el segmento .data justo después de la ejecución, se obtuvo lo siguiente para cada ejercicio:

#### Sensor de luminosidad

| Registro   | Dirección asignada | Valor tras ejecutar (caso de prueba) |
|------------|--------------------|--------------------------------------|
| LuzControl | 0x10010000         | 0x00000001                           |
| LuzEstado  | 0x10010004         | 0x00000001                           |
| LuzDatos   | 0x10010008         | 0x000002bc (700 en decimal)          |

### Módulo de tensión arterial

| Registro       | Dirección asignada | Valor tras ejecutar (caso de prueba) |
|----------------|--------------------|--------------------------------------|
| TensionControl | 0x10010000         | 0x00000001                           |
| TensionEstado  | 0x10010004         | 0x00000001                           |
| TensionSistol  | 0x10010008         | 0x00000078 (120)                     |
| TensionDiastol | 0x1001000c         | 0x00000050 (80)                      |

Lo obtenido confirma que el ensamblador reserva las tres o cuatro palabras de cada dispositivo en direcciones consecutivas, alineadas a palabra, respetando estrictamente el orden en que se declararon con `.word` dentro de `.data`, sin dejar huecos entre ellas ni solaparlas con ninguna otra variable. También se puede notar algo relevante, ambos programas arrancan su mapa de memoria exactamente en la misma dirección base, `0x10010000`, porque cada `.asm` se ensambla como un programa independiente y aislado.

Si en algún momento se quisiera integrar el sensor de luminosidad y el módulo de tensión arterial dentro de un mismo sistema físico, compartiendo un solo procesador y un solo mapa de direcciones, sería obligatorio reubicar uno de los dos bloques de registros, por ejemplo desplazando `TensionControl` a partir de `0x10010010` en adelante, porque de lo contrario ambos dispositivos terminarían en la misma región de memoria, con cuatro registros de tensión intentando convivir con tres registros de luminosidad en el mismo rango de direcciones.

### b. Verificación del protocolo de comunicación control-estado-datos.

Aunque los dos códigos resuelven problemas distintos, siguen exactamente el mismo patrón de tres fases que es el estándar en el manejo de dispositivos por memoria mapeada mediante sondeo, y que corresponde a lo que el libro describe como el patrón clásico de driver por polling con bit de listo. La primera fase es la orden de inicio, escribiendo un 1 en el registro de control mediante `sw`, tanto en `InicializarSensorLuz` sobre `LuzControl` como en `procesarMedicion` sobre `TensionControl`. La segunda fase es la espera activa sobre el registro de estado, con `lw` seguido de `beq` dentro de la etiqueta `espera` en ambos programas, que revisa continuamente la misma

dirección de memoria hasta que el valor deje de ser cero.

El último paso es la recuperación del resultado, realizada con un lw final sobre el registro o registros de datos, LuzDatos en un caso, TensionSistol y TensionDiastol en el otro. Se constató que ambos algoritmos respetan este orden, ninguno intenta leer el registro de datos antes de haber confirmado que el estado indica que hay una lectura disponible, lo cual es fundamental en memory-mapped I/O porque, a diferencia de una variable común, el valor de un registro de datos de un dispositivo no tiene ningún significado válido hasta que el propio dispositivo así lo confirma a través de su registro de estado.

### **c. Interpretación de los distintos estados.**

El enunciado de cada ejercicio define explícitamente qué significa cada valor posible del registro de estado, y la parte más relevante del análisis consiste en comprobar, forzando cada uno de esos valores directamente en la ventana de memoria de datos antes de ejecutar, que el algoritmo reacciona correctamente ante cada uno de ellos.

Para el sensor de luminosidad, se comprobó que con LuzEstado en 1 el programa abandona correctamente el bucle espera y continúa su ejecución con normalidad, sin importar si el valor de LuzDatos corresponde al extremo inferior permitido, 0, o al extremo superior, 1023, de modo que LeerLuminosidad no impone ninguna restricción indebida sobre el dato leído más allá de comprobar el estado del sensor. También, forzando LuzEstado a -1 antes de que se ejecutara InicializarSensorLuz, se pudo observar que el algoritmo reconoce correctamente el código de error de hardware, saliendo del bucle de espera, comparando el valor leído con -1 mediante li \$t3, -1 y beq \$t2, \$t3, errorHardware, y devolviendo -1 en \$v0.

Adicionalmente, se experimentó con un tercer escenario, en el que la inicialización se completa con éxito, LuzEstado vale 1 en ese momento, pero el sensor reporta una falla justo antes de que se invoque LeerLuminosidad, cambiando su estado a -1, algo posible en un dispositivo real si ocurre una interferencia entre el momento en que se anuncia listo y el momento en que efectivamente se transfiere el dato. Se confirmó que LeerLuminosidad también considera este caso de forma independiente, revisando otra vez LuzEstado al inicio de su propia ejecución, y

devolviendo el código de error de lectura, -1 en \$v1, sin arrastrar ningún dato inválido hacia \$v0.

El sensor de luminosidad tal como se implementó conserva los tres estados requeridos, 0 como no listo, 1 como lectura disponible, y -1 como error de hardware, y de la misma manera distingue correctamente entre un error ocurrido durante la inicialización y uno ocurrido durante la lectura, devolviendo la información por la vía adecuada en cada caso, ya sea a través del valor de retorno de InicializarSensorLuz o a través de \$v1 en LeerLuminosidad.

Para el módulo de tensión arterial, al tener TensionEstado en 1, el algoritmo abandona correctamente el ciclo espera y recupera tanto TensionSistol como TensionDiastol sin cambio alguno, tanto para valores típicos como 120 sobre 80, como para valores en los extremos de una medición real, por ejemplo 90 sobre 60 en el extremo bajo o 220 sobre 140 en el extremo alto. La etiqueta procesarMedicion trata ambos registros de datos exactamente igual sin imponer ningún filtro sobre el rango fisiológico del resultado, ya que la validación de si esos valores son clínicamente razonables no es responsabilidad del controlador del dispositivo sino de una capa superior del software.

Aquí sin embargo, también se puede notar la limitación más importante de este segundo ejercicio en comparación con el primero. El enunciado de tensión arterial solamente define dos estados posibles para TensionEstado, 0 mientras mide y 1 cuando el resultado está listo, sin reservar ningún valor para señalar un error de hardware, y el código de procesarMedicion manifiesta esa limitación, dado que su espera solamente sabe reconocer una condición de salida, que el valor deje de ser cero, sin ningún paso extra capaz de distinguir entre un dispositivo que está midiendo normalmente y uno que está fallando.

No es un error de implementación porque el algoritmo cumple exactamente lo que se pidió, pero sí demuestra una diferencia en la robustez del protocolo de comunicación entre ambos dispositivos, puesto que un protocolo de memoria mapeada es tan robusto como los estados que su propio diseño abarca, notándose en la práctica que reservar un valor exclusivo para error, como sí se hizo con -1 en el sensor de luminosidad, es una decisión de diseño que vale la pena repetir en cualquier dispositivo mapeado en memoria que se plantee en un futuro.

#### **d. Comprobación del riesgo de espera activa indefinida.**

Asimismo, se puso a prueba intencionalmente la situación en el que el registro de estado nunca cambia de valor, dejando LuzEstado y TensionEstado fijos en 0 durante toda la simulación, para corroborar lo que se explicó de manera teórica sobre los riesgos de la espera activa. Tanto en un caso como en el otro, los programas quedaron atrapados de forma permanente dentro del bucle espera, ejecutando indefinidamente el mismo par de instrucciones lw y beq sobre la misma dirección de memoria.

La consulta continua que exige la memoria mapeada sin interrupciones no tiene ninguna forma de distinguir entre un dispositivo que simplemente está tardando en responder y otro que jamás va a responder, y ese es precisamente el motivo por el cual en un sistema este tipo de protocolo suele complementarse con las técnicas de interrupciones, DMA o límites de tiempo, ninguna de las cuales estaba dentro del alcance de esta práctica, pero su necesidad queda demostrada por el comportamiento observado en ambos algoritmos.

Los resultados obtenidos aseguran que los algoritmos cumplen adecuadamente con el objetivo central de esta práctica, manipular de forma adecuada un conjunto de registros mapeados en memoria, sin necesitar ninguna instrucción especial de entrada o salida, respetando en todo momento las tres fases de control, estado y datos, y reaccionando de forma correcta ante cada uno de los estados que el enunciado de cada dispositivo describe. La diferencia más significativa que dejó el análisis comparativo entre ambos ejercicios no tiene que ver con su rendimiento, sino con el diseño de comunicación entre el procesador y el dispositivo.

El sensor de luminosidad, al contar con un valor de estado reservado para error de hardware, resulta en un protocolo de memoria mapeada más completo y más fácil de depurar ante una falla que pueda ocurrir, mientras que el módulo de tensión arterial, siguiendo estrictamente lo que su enunciado especifica, puede quedar bloqueado sin ningún indicio de que algo salió mal, conectando con todo lo discutido en el resto de preguntas sobre la organización, las ventajas y los riesgos de la entrada y salida mapeada en memoria.